

Standard & Campaign Bonuses and Wagering

A developer's guide to bonus configuration and wagering-requirement logic for the iGaming platform

1 What is Wagering? A Beginner's Guide

Before diving into the code, you must understand the business logic of a casino bonus.

The Core Concept

A bonus is NOT free money to play with. A bonus is a reward that the player earns by playing enough games with their own real money.

Think of it like a loyalty reward at a coffee shop:

“Buy 10 coffees (with your own money), and the 11th one is free.”

In our system it works exactly the same way:

“Place ₹2,00,000 worth of bets (with your own deposited money), and we will credit ₹10,000 bonus to your balance.”

The Flow (Step by Step)

- 1** Player Deposits Real Money — e.g., ₹10,000 goes into their main balance.
- 2** Player Claims a Bonus — The system creates a bonus record in the database. The bonus amount (e.g., ₹10,000) is NOT added to their balance yet. It is stored as a pending reward with a wagering target.
- 3** Player Plays Games Using Their Own Real Money — Every bet they place (₹100 spin, ₹500 hand, etc.) is tracked and added to `wagering_completed`.
- 4** Wagering Target is Met — Once `wagering_completed` $\geq$ `wagering_required`, the system marks the bonus as “Cleared”.
- 5** Bonus Amount is Credited — Only NOW does the bonus amount get added to the player's real withdrawable balance.

Important Rules for Developers

- ✗ The player does NOT play with the bonus money.
- ✓ The player plays with their own deposited (real) money.
- ✓ The bonus is a future reward for meeting the wagering target.
- ✓ We track the total amount bet (not wins or losses). Every rupee wagered counts toward the goal.

2 Standard / Campaign Bonuses (The Configuration)

In our system, standard and campaign bonuses are configurable promotions managed via the admin panel (stored in `tbl_bonuses`). As a developer, you will be checking these rules to see if a player qualifies for a bonus.

Key Database Configurations

Deposit Sequence Restrictions

- `is_first_deposit`: Triggers only on the player's very first lifetime deposit.
- `is_second_deposit` / `is_third_deposit`: Specifically targets returning players making their 2nd or 3rd deposit.

Player Type Filters

- `is_new_player_only`: Restricts the bonus to freshly registered accounts.

Bonus Scope

- Mass: Available to all eligible players on the platform.
- Single Account (Targeted): Issued directly to a specific user via a unique code.

Redemption Logic

- Percentage-Based: (e.g., 100% Match Bonus) If a user deposits ₹5,000, the pending bonus reward is ₹5,000.
- Fixed Amount: (e.g., "Deposit ₹2,000, Get ₹1,000") Regardless of depositing ₹2,000 or ₹10,000, the pending bonus reward is a flat ₹1,000.
- Max Redeem Value: Caps the maximum bonus (e.g., "100% Match up to ₹50,000").

3 The Wagering Condition Equations

When you are writing the backend logic to track a player's progress, here is the math you need to implement.

The Core Equation

$$\text{Wagering Required} = \text{Bonus Amount} \times \text{Wagering Multiplier}$$

The player must bet this much real money before they receive the bonus.

Example 1: Standard First Deposit Bonus (Slots)

Scenario: A player claims a "100% First Deposit Bonus" for Slot games.

Field	Value
Player Deposits	₹10,000 (goes to real balance)
Pending Bonus Reward	₹10,000 (stored in DB, NOT in balance)

Field	Value
Multiplier for Slots	20x

Step-by-step Developer Logic

Calculate target: ₹10,000 (Bonus) × 20 = ₹2,00,000.

Save to database:

```
bonus_amount = 10000
wagering_required = 200000
wagering_completed = 0
status = 'active'
```

Player plays slots using their ₹10,000 real balance. Every ₹100 spin updates:

```
wagering_completed = wagering_completed + 100
```

When wagering_completed >= 200000:

- Update status = 'completed'
- Credit ₹10,000 bonus to tblusersdata.tbl_balance
- The player can now withdraw this ₹10,000.

Example 2: Provider-Specific Weighting (Risk Balancing)

Not all games are created equal. Slot machines are high-risk for players (house wins more often), but Live Casino games like Baccarat are almost a 50/50 coin flip. If the casino allowed 20x wagering on Baccarat, the player could grind through the requirement easily without losing much money, then collect the bonus for almost free.

To fix this, the system uses tbl_bonus_providers to set different multipliers for different game providers.

Scenario: A player claims a ₹5,000 Bonus.

Provider	Multiplier	Target
Provider A (Jili – Slots)	15x	₹75,000
Provider B (Evolution – Live Casino)	50x	₹2,50,000

The Developer Logic

- When the bet API callback says the player bet ₹1,000 on Provider A (Slots), look up the multiplier (15x). The target is ₹75,000. Add ₹1,000 to wagering_completed.
- When the bet API callback says the player bet ₹1,000 on Provider B (Live Casino), look up the multiplier (50x). The target is ₹2,50,000. Add ₹1,000 to wagering_completed.
- Once the target for the active provider rules is met, credit the ₹5,000 bonus to the player’s real balance.